

3.<Źródło>Algorytmy optymalizacji

Zgodnie z Pańskim poleceniem, przedstawiam szczegółowy opis zagadnień programowania liniowego, całkowitoliczbowego i binarnego w kontekście algorytmów optymalizacji, bazując na dostarczonym materiale.

Raport: Zaawansowane Techniki Optymalizacji

Spis treści

1. [Wprowadzenie do Optymalizacji](#)
 2. [Programowanie Liniowe \(PL\)](#)
 - [2.1 Sformułowanie Problemu i Rozwiązywalność](#)
 - [2.2 Metoda Sympleks](#)
 - [2.3 Przykład Ilustracyjny PL](#)
 3. [Programowanie Liniowe Całkowitoliczbowe \(PLC\)](#)
 - [3.1 Metoda Odcięć](#)
 - [3.2 Metoda Przeglądu \(Algorytm Land-Doiga\)](#)
 4. [Programowanie Liniowe Binarne \(PLB\)](#)
 - [4.1 Algorytm Balasa \(Metoda Enumeracji Niejawnej\)](#)
 - [4.2 Przykład Ilustracyjny PLB](#)
 5. [Podsumowanie](#)
-

1. Wprowadzenie do Optymalizacji

Optymalizacja jest dziedziną matematyki stosowanej, której celem jest znalezienie najlepszego (optymalnego) rozwiązania z dostępnego zbioru możliwych rozwiązań. Formalnie, problem minimalizacji skalarnej można zapisać jako:

$$K(x^*) = \min_{x \in X} K(x)$$

gdzie:

- $K(x)$ to **funkcja celu**, którą dążymy do zminimalizowania.
- x to **wektor zmiennych decyzyjnych** (rozwiązanie).
- x^* to **rozwiązanie optymalne**.
- X to **zbiór rozwiązań dopuszczalnych**, określony przez zestaw warunków ograniczających.

Warto zauważyć, że problem maksymalizacji można łatwo przekształcić w problem minimalizacji, korzystając z tożsamości:

$$\max_{x \in X} K(x) = - \min_{x \in X} -K(x)$$

W ramach niniejszej analizy skupiamy się na przypadkach, gdzie funkcja celu i ograniczenia mają charakter liniowy, a zmienne decyzyjne mogą przyjmować wartości ciągłe, całkowitoliczbowe lub binarne.

2. Programowanie Liniowe (PL)

2.1 Sformułowanie Problemu i Rozwiązywalność

Programowanie liniowe (PL) dotyczy problemów, w których zarówno funkcja celu, jak i ograniczenia są funkcjami liniowymi. Standardowa postać problemu PL to:

$$\max_x x_0 = c^T x$$

przy ograniczeniach:

$$Ax = b$$

$$x \geq 0$$

gdzie:

- c jest wektorem współczynników funkcji celu.
- x jest wektorem zmiennych decyzyjnych.

- A jest macierzą współczynników ograniczeń.
- b jest wektorem prawych stron ograniczeń.

Dla każdego zadania PL może wystąpić jeden z trzech przypadków:

1. **Problem sprzeczny:** Brak rozwiązań dopuszczalnych ($X = \emptyset$).
2. **Problem nieograniczony:** Funkcja celu może przyjmować dowolnie duże wartości.
3. **Problem posiada rozwiązanie optymalne:** Istnieje co najmniej jedno rozwiązanie x^* , dla którego funkcja celu osiąga wartość maksymalną.

Dalsza analiza koncentruje się na przypadku trzecim.

2.2 Metoda Sympleks

Metoda sympleks jest iteracyjnym algorytmem służącym do rozwiązywania problemów PL. Jej działanie opiera się na przechodzeniu pomiędzy wierzchołkami wielościanu dopuszczalnych rozwiązań w taki sposób, aby w każdym kroku poprawiać wartość funkcji celu.

Koncepcja Rozwiązania Bazowego: Algorytm wykorzystuje pojęcie **rozwiązania bazowego**. Zmienne x i kolumny macierzy A dzieli się na dwie grupy: bazowe (B) i niebazowe (N).

- $A = [B, N]$
- $x = [x_B, x_N]^T$

Rozwiązanie bazowe uzyskuje się, ustawiając zmienne niebazowe na zero ($x_N = 0$), co prowadzi do:

$$x_B = B^{-1}b$$

Jeśli dodatkowo $x_B \geq 0$, rozwiązanie nazywane jest **dopuszczalnym rozwiązaniem bazowym**. Kluczowe twierdzenie programowania liniowego mówi, że jeśli istnieje rozwiązanie optymalne, to istnieje również dopuszczalne rozwiązanie bazowe, które jest optymalne.

Kroki Algorytmu Sympleks:

1. **Start:** Znajdź początkowe dopuszczalne rozwiązanie bazowe.
2. **Test optymalności:** Oblicz tzw. **koszty zredukowane** dla zmiennych niebazowych. W postaci tablicowej są to współczynniki y_{0j} w wierszu funkcji celu. Jeśli wszystkie $y_{0j} \geq 0$ (dla problemu maksymalizacji), bieżące rozwiązanie jest optymalne. Koniec algorytmu.
3. **Wybór zmiennej wchodzącej:** Wybierz zmienną niebazową x_k , która ma największy potencjał poprawy funkcji celu (np. najbardziej ujemny współczynnik y_{0k}). Ta zmienna wejdzie do bazy.
4. **Wybór zmiennej wychodzącej:** Zwiększaj wartość x_k do momentu, aż jedna ze zmiennych bazowych x_{B_r} osiągnie wartość 0. Zmienna ta jest wyznaczana przez tzw. **test ilorazowy**:

$$v_{rk} = \min_i \left\{ \frac{y_{i0}}{y_{ik}} : y_{ik} > 0 \right\}$$

Zmienna x_{B_r} opuszcza bazę.

5. **Pivot (operacja obrotu):** Zaktualizuj tablicę sympleksową, aby odzwierciedlić nową bazę. Wróć do kroku 2.

2.3 Przykład Ilustracyjny PL

Rozważmy problem:

$$\max(3x_1 + 2x_2)$$

przy ograniczeniach:

$$\begin{cases} 2x_1 + 3x_2 \leq 24 \\ 14x_1 + 6x_2 \leq 84 \\ x_1 \leq 5 \\ x_2 \leq 7 \\ x_1, x_2 \geq 0 \end{cases}$$

Po dodaniu zmiennych luzu (x_3, x_4, x_5, x_6) i zastosowaniu metody sympleks w kolejnych iteracjach (przedstawionych w tablicach na slajdach 15-18), algorytm dochodzi do rozwiązania optymalnego.

Wynik (z iteracji czwartej, slajd 18):

- Wartość funkcji celu: $x_0 = 22$.
- Wszystkie współczynniki w wierszu zerowym (y_{0j}) są nieujemne, co potwierdza optymalność.
- Zmienne bazowe i ich wartości: $x_5 = 1.2/5$, $x_2 = 5.3/5$, $x_1 = 3.3/5$, $x_6 = 1.2/5$.
- Rozwiązanie w przestrzeni zmiennych oryginalnych: $x_1 = 3.6$, $x_2 = 5.6$. Rozwiązanie to jest zgodne z interpretacją graficzną (slajd 14).

3. Programowanie Liniowe Całkowitoliczbowe (PLC)

Problem PLC jest rozszerzeniem PL, w którym na zmienne decyzyjne nałożony jest dodatkowy warunek całkowitoliczbowości:

$$x - \text{całkowite}$$

Do rozwiązywania problemów PLC stosuje się dwie główne metody, które wykorzystują rozwiązanie "zrelaksowanego" problemu PL (bez warunku całkowitoliczbowości).

3.1 Metoda Odcięć

Idea tej metody (np. algorytm Gomory'ego) polega na iteracyjnym dodawaniu nowych ograniczeń, zwanych **odcięciami**.

Procedura:

1. Rozwiąż problem PL (relaksację).
2. Jeśli rozwiązanie jest całkowitoliczbowe, jest ono optymalne dla PLC. Koniec.
3. Jeśli nie, wybierz wiersz w końcowej tablicy sympleksowej, w którym zmienna bazowa ma wartość niecałkowitą:

$$x_{B_i} + \sum_{j \in R} y_{ij} x_j = y_{i0}$$

4. Wygeneruj **odcięcie Gomory'ego**, które "odcina" bieżące niecałkowitoliczbowe rozwiązanie, ale nie eliminuje żadnego dopuszczalnego rozwiązania całkowitoliczbowego. Odcięcie to ma postać:

$$\sum_{j \in R} f_{ij} x_j \geq f_{i0}$$

gdzie $f_{ij} = y_{ij} - \lfloor y_{ij} \rfloor$ oraz $f_{i0} = y_{i0} - \lfloor y_{i0} \rfloor$ oznaczają części ułamkowe.

5. Dodaj nowe ograniczenie (odcięcie) do problemu i wróć do kroku 1.

Graficznie (slajd 25), odcięcie to nowa hiperpłaszczyzna, która zacieśnia zbiór rozwiązań dopuszczalnych.

3.2 Metoda Przeglądu (Algorytm Land-Doiga)

Jest to metoda z rodziny algorytmów **"podziału i ograniczeń"** (Branch and Bound, B&B).

Procedura:

1. **Inicjalizacja:** Ustaw górne ograniczenie na koszt $UB = \infty$.
2. **Podział (Branch):** Rozwiąż relaksację PL bieżącego problemu.
 - Jeśli rozwiązanie jest całkowitoliczbowe i lepsze niż obecne UB , zaktualizuj UB i zapisz to rozwiązanie jako najlepsze dotychczas.
 - Jeśli rozwiązanie jest niecałkowitoliczbowe, wybierz zmienną x_k , która przyjęła wartość niecałkowitą $\tilde{x}_k$. Stwórz dwa nowe podproblemy (gałęzie drzewa poszukiwań), dodając odpowiednio ograniczenia:
 - Gałąź 1: $x_k \leq \lfloor \tilde{x}_k \rfloor$
 - Gałąź 2: $x_k \geq \lfloor \tilde{x}_k \rfloor + 1$
3. **Ograniczenie (Bound):** W każdej gałęzi:
 - Jeśli koszt rozwiązania relaksacji PL jest gorszy niż bieżące UB , odrzuć tę gałąź (nie może zawierać lepszego rozwiązania).
 - Jeśli podproblem jest sprzeczny, odrzuć gałąź.
4. Rekursywnie powtarzaj proces dla aktywnych gałęzi, aż całe drzewo zostanie przeszukane.

Drzewo na slajdzie 28 ilustruje ten proces podziału na przykładzie, gdzie problem jest dzielony ze względu na zmienne x_1 i x_2 .

4. Programowanie Liniowe Binarne (PLB)

PLB jest szczególnym przypadkiem PLC, w którym zmienne mogą przyjmować jedynie wartości 0 lub 1.

$$x \in \{0, 1\}^n$$

4.1 Algorytm Balasa (Metoda Enumeracji Niejawnej)

To wyspecjalizowany algorytm typu B&B dla problemów PLB, który nie wymaga rozwiązywania relaksacji PL w każdym węźle. Zamiast tego wykorzystuje testy logiczne do **niejawnego przeglądania** (eliminowania) dużych grup rozwiązań.

Założenia: Wszystkie współczynniki funkcji celu $c_j \geq 0$ (dla problemu minimalizacji). Jeśli $c_j < 0$, dokonuje się substytucji $x'_j = 1 - x_j$.

Procedura: Algorytm przeszukuje drzewo binarne, w którym gałęzie odpowiadają ustaleniu wartości zmiennej na 0 lub 1. W każdym węźle (charakteryzowanym przez zbiór J zmiennych o ustalonych wartościach) wykonywane są testy w celu odcięcia (pruning) poddrzewa:

1. **Test poprawy funkcji celu:** Sprawdza, czy istnieje jeszcze szansa na znalezienie rozwiązania lepszego niż najlepsze dotychczasowe (UB). Węzeł jest eliminowany, jeśli dla każdej nieustalonej zmiennej j zachodzi $z + c_j \geq UB$, gdzie z to koszt częściowego rozwiązania.
2. **Test niedopuszczalności:** Sprawdza, czy którekolwiek z ograniczeń jest niemożliwe do spełnienia, nawet po ustawieniu wszystkich "pomocnych" zmiennych na 1. Jeśli tak, węzeł jest eliminowany.
3. **Test rozgałęzień:** Jeśli węzeł nie został wyeliminowany, wybiera się zmienną do dalszego podziału. Zazwyczaj wybiera się tę, która w największym stopniu przyczynia się do "naprawy" niespełnionych ograniczeń.

4.2 Przykład Ilustracyjny PLB

Rozważmy problem (slajd 35):

$$\min(5x_1 + 7x_2 + 10x_3 + 3x_4 + x_5)$$

przy ograniczeniach:

$$\begin{cases} -x_1 + 3x_2 - 5x_3 - x_4 + 4x_5 \leq -2 \\ 2x_1 - 6x_2 + 3x_3 + 2x_4 - 2x_5 \leq 0 \\ x_2 - 2x_3 + x_4 + x_5 \leq -1 \\ x_i \in \{0, 1\} \end{cases}$$

Przebieg algorytmu (slajd 36) pokazuje kolejne kroki, w których ustalane są wartości zmiennych, a zbiory rozwiązań są analizowane i odrzucane. Drzewo na slajdzie 37 wizualizuje ścieżkę poszukiwań. Algorytm bada kolejne węzły (1, 2, 3, 4, 5), modyfikując częściowe rozwiązania i odcinając gałęzie, które nie mogą prowadzić do optymalnego wyniku.

5. Podsumowanie

Przedstawione techniki stanowią fundament optymalizacji dyskretniej i ciągłej.

- **Programowanie Liniowe i metoda sympleks** dostarczają efektywnego (w praktyce) narzędzia do rozwiązywania problemów z liniową funkcją celu i ograniczeniami, należących do klasy złożoności P.
- **Programowanie Całkowitoliczbowe i Binarne** znacząco rozszerzają zakres modelowania, pozwalając na uwzględnienie warunków dyskretnych. Problemy te są jednak co do zasady NP-trudne. Metody takie jak **odcięcia i podział i ograniczenia (B&B)** stanowią podstawowe podejścia do ich rozwiązywania, balansując między dokładnością a czasem obliczeń. Algorytmy specjalizowane, jak **algorytm Balasa**, wykorzystują specyficzną strukturę problemu binarnego do efektywniejszego przeszukiwania przestrzeni rozwiązań.